Name: Digital LED Hourglass (Sandclock)

Description:

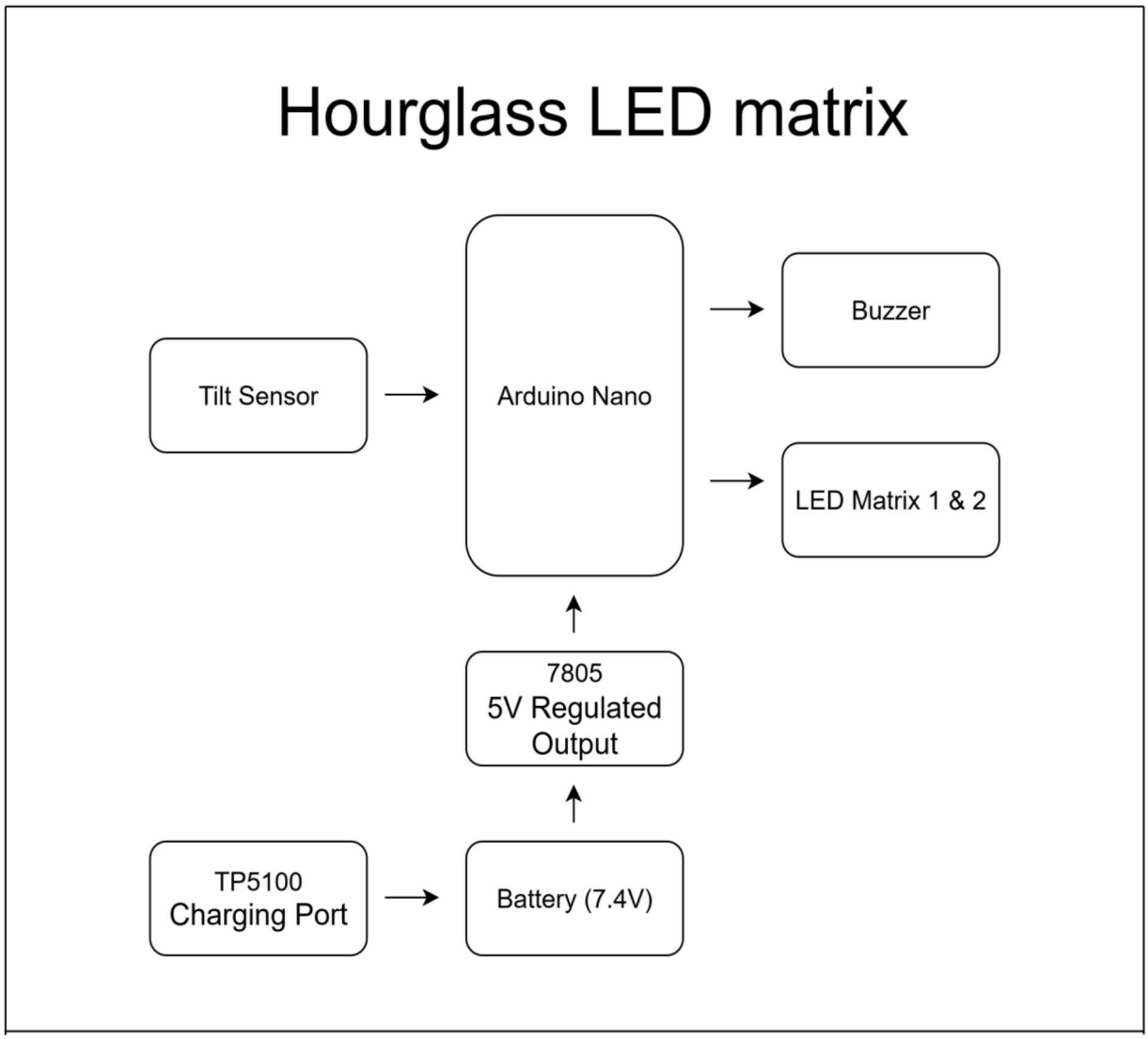
This project simulates an **hourglass** using two 8×8 LED matrices that visually represent sand flowing from one side to another when the device is tilted. A **tilt sensor** detects the change in orientation, and the display dynamically reverses direction, accompanied by **buzzer sounds** for start, flow, and completion. The system is powered by a rechargeable battery via a **TP5100 charging module** and regulated by a **7805 voltage regulator**.

Materials Used:

S.NO	Component	Description
1	Arduino Nano	Microcontroller that controls the LED matrices and tilt logic.
2	8×8LED Matrix	Displays sand particles that flow visually like an hourglass.
3	Tilt Sensor (SW-520D)	Detects the tilt or rotation of the hourglass.
4	Buzzer	Provides audio feedback for start, flow, and completion events.
5	TP5100 Charging Module	Used to charge the lithium-ion battery safely.
6	7805 Voltage Regulator	Regulates voltage to 5V for Arduino and LED matrices.
7	Lithium-ion Battery (3.7V)	Powers the device through the 7805 regulator.
8	Slide Switch	Used to power ON/OFF the device.

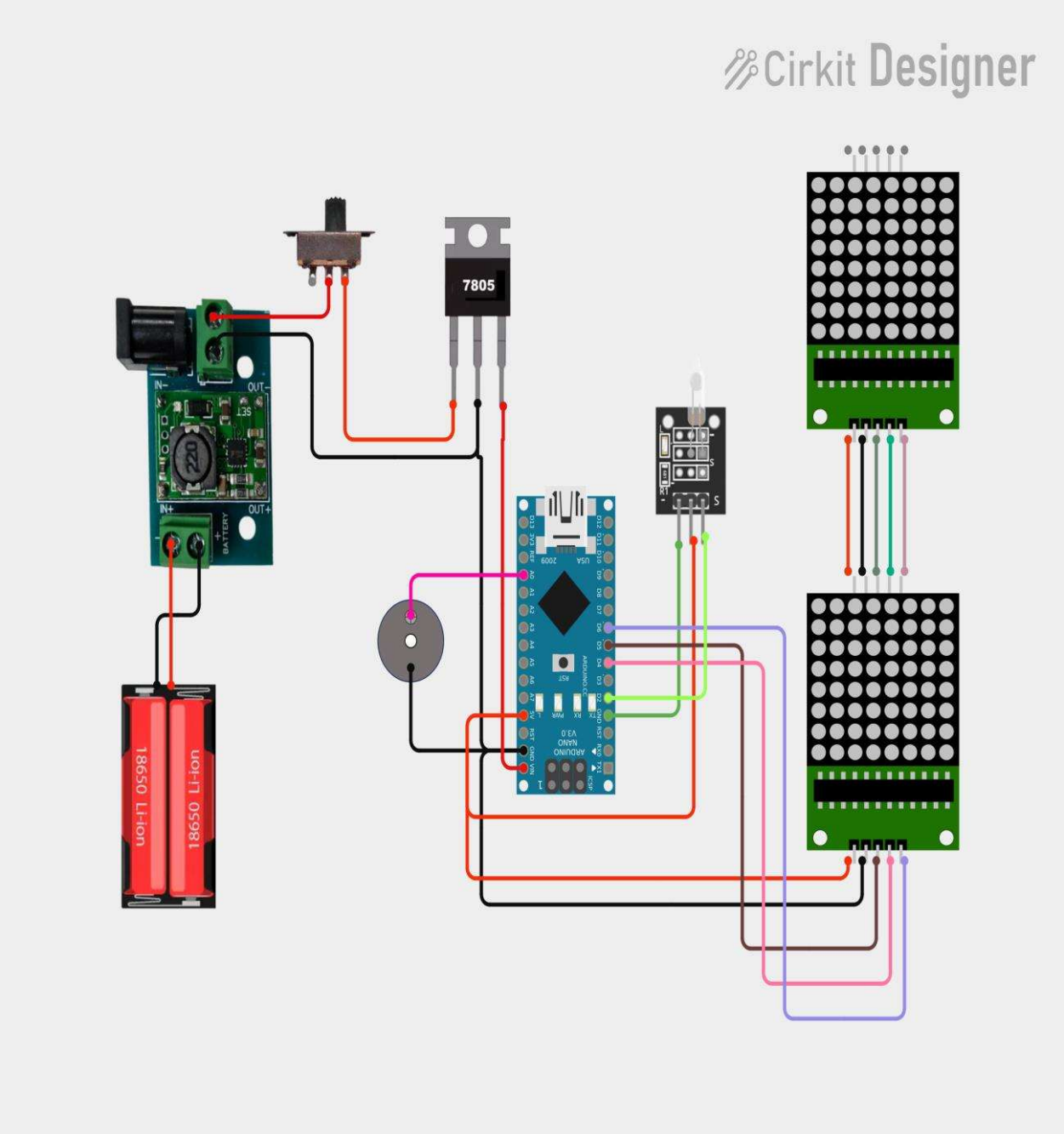
Copyright 2025 YuviPep© All Rights Reserved

Block Diagram:



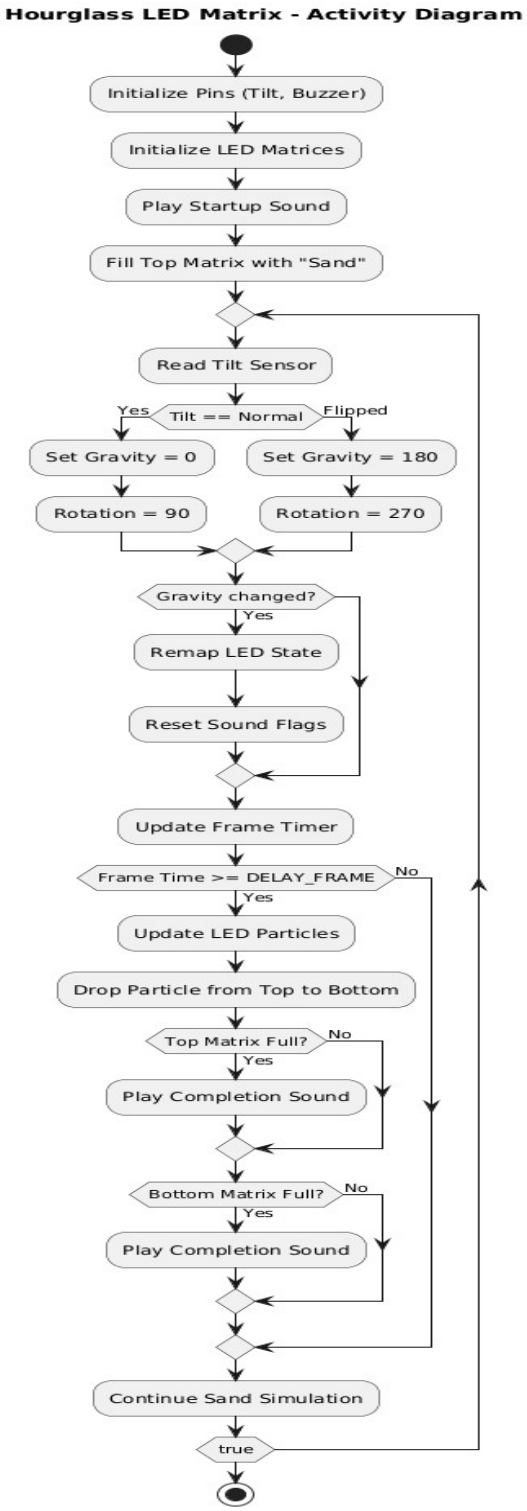
Circuit Diagram:

Copyright 2025 YuviPep© All Rights Reserved



Copyright 2025 YuviPep© All Rights Reserved

Flow Chart:



Copyright 2025 YuviPep© All Rights Reserved

How to Explain the Project:

- This is a **digital version of an hourglass**, built using **LED matrices**.
- When the device is upright, the LEDs simulate sand filling up on one side.
- When tilted, the **tilt sensor detects** the change and reverses the animation direction.
- The **buzzer sounds** at the start, during flow, and when one side fills completely.
- It's powered by a **rechargeable battery** charged via a **TP5100** module.
- Demonstrates **matrix animation, sensor detection, and sound interaction** with microcontroller control.

Code:

```
#include <LedControl.h>

// Matrix device indices
#define MATRIX_TOP 1
#define MATRIX_BOTTOM 0

// Pin Definitions
#define PIN_DATAIN 5
#define PIN_CLK 4
#define PIN_LOAD 6
#define PIN_BUZZER 14
#define PIN_TILT 2 // Tilt sensor input

#define DELAY_FRAME 50      // Faster matrix updates
#define DELAY_DROP 600      // Faster sand drop

#define ROTATION_OFFSET 90

LedControl lc = LedControl(PIN_DATAIN, PIN_CLK, PIN_LOAD, 2);

bool ledState[2][8][8] = {{{false}}};
```

Copyright 2025 YuviPep© All Rights Reserved

```
int currentGravity = 0;
int current_rotation = 0; // 0 or 180 only

struct coord { byte x, y; };

coord getPhysicalCoord(int rot, byte x, byte y) {
  switch (rot % 360) {
    case 0: return {x, y};
    case 90: return {y, (byte)(7 - x)};
    case 180: return {(byte)(7 - x), (byte)(7 - y)};
    case 270: return {(byte)(7 - y), x};
    default: return {x, y};
  }
}

coord getLogicalCoord(int rot, byte x, byte y) {
  return getPhysicalCoord((360 - rot) % 360, x, y);
}

void setLed(int device, int x, int y, bool val) {
  coord phys = getPhysicalCoord(current_rotation, (byte)x, (byte)y);
  lc.setLed(device, phys.y, phys.x, val);
  ledState[device][x][y] = val;
}

bool getLed(int device, int x, int y) {
  return ledState[device][x][y];
}

void remapState(int old_rot, int new_rot) {
  bool newState[2][8][8] = {{{false}}};
  for (int device = 0; device < 2; device++) {
    for (byte lx = 0; lx < 8; lx++) {
      for (byte ly = 0; ly < 8; ly++) {
        coord phys = getPhysicalCoord(new_rot, lx, ly);
        coord old_l = getLogicalCoord(old_rot, phys.x, phys.y);
        newState[device][lx][ly] = ledState[device][old_l.x][old_l.y];
      }
    }
  }
  memcpy(ledState, newState, sizeof(newState));
  // Sync hardware with new state
```

Copyright 2025 YuviPep© All Rights Reserved

```
  for (int device = 0; device < 2; device++) {
    for (byte x = 0; x < 8; x++) {
      for (byte y = 0; y < 8; y++) {
        coord phys = getPhysicalCoord(new_rot, x, y);
        lc.setLed(device, phys.y, phys.x, newState[device][x][y]);
      }
    }
  }
}

coord getDown(int x, int y) { return { (byte)(x - 1), (byte)(y + 1) }; }
coord getLeft(int x, int y) { return { (byte)(x - 1), y }; }
coord getRight(int x, int y) { return { x, (byte)(y + 1) }; }

bool canGoLeft(int device, int x, int y) {
  if (x == 0) return false;
  coord l = getLeft(x, y);
  return !getLed(device, l.x, l.y);
}

bool canGoRight(int device, int x, int y) {
  if (y == 7) return false;
  coord r = getRight(x, y);
  return !getLed(device, r.x, r.y);
}

bool canGoDown(int device, int x, int y) {
  if (y == 7 || x == 0) return false;
  coord d = getDown(x, y);
  return !getLed(device, d.x, d.y);
}

void goDown(int device, int x, int y) {
  setLed(device, x, y, false);
  coord d = getDown(x, y);
  setLed(device, d.x, d.y, true);
}

void goLeft(int device, int x, int y) {
  setLed(device, x, y, false);
  coord l = getLeft(x, y);
  setLed(device, l.x, l.y, true);
```

Copyright 2025 YuviPep© All Rights Reserved

```
}

void goRight(int device, int x, int y) {
  setLed(device, x, y, false);
  coord r = getRight(x, y);
  setLed(device, r.x, r.y, true);
}

bool moveParticle(int device, int x, int y) {
  if (!getLed(device, x, y)) return false;

  bool can_GoLeft = canGoLeft(device, x, y);
  bool can_GoRight = canGoRight(device, x, y);
  bool can_GoDown = canGoDown(device, x, y);

  if (can_GoDown) {
    goDown(device, x, y);
  } else if (can_GoLeft && !can_GoRight) {
    goLeft(device, x, y);
  } else if (can_GoRight && !can_GoLeft) {
    goRight(device, x, y);
  } else if (can_GoLeft && can_GoRight) {
    if (random(2) == 0) goLeft(device, x, y);
    else goRight(device, x, y);
  } else {
    return false;
  }
  return true;
}

void fill(int device, int maxcount) {
  int n = 8;
  int count = 0;
  for (int slice = 0; slice < 2 * n - 1; ++slice) {
    int z = slice < n ? 0 : slice - n + 1;
    for (int j = z; j <= slice - z; ++j) {
      byte y = 7 - j;
      byte x = slice - j;
      if (++count <= maxcount) {
        setLed(device, x, y, true);
      }
    }
```

Copyright 2025 YuviPep© All Rights Reserved

```
  }
}

int countParticles(int device) {
  int c = 0;
  for (byte x = 0; x < 8; x++)
    for (byte y = 0; y < 8; y++)
      if (getLed(device, x, y)) c++;
  return c;
}

bool updateMatrix() {
  bool moved = false;
  int n = 8;
  for (int slice = 0; slice < 2 * n - 1; ++slice) {
    bool direction = (random(2) == 0);
    int z = slice < n ? 0 : slice - n + 1;
    for (int j = z; j <= slice - z; ++j) {
      byte y = direction ? (7 - j) : (7 - (slice - j));
      byte x = direction ? (slice - j) : j;
      if (moveParticle(MATRIX_BOTTOM, x, y)) moved = true;
      if (moveParticle(MATRIX_TOP, x, y)) moved = true;
    }
  }
  return moved;
}

bool dropParticle() {
  static unsigned long lastDrop = 0;
  static bool flowStarted = false;
  unsigned long now = millis();
  if (now - lastDrop < DELAY_DROP) return false;
  lastDrop = now;

  if (currentGravity != 0 && currentGravity != 180) return false;

  coord a_pos = getLogicalCoord(current_rotation, 0, 0);
  coord b_pos = getLogicalCoord(current_rotation, 7, 7);

  bool a_on = getLed(MATRIX_TOP, a_pos.x, a_pos.y);
  bool b_on = getLed(MATRIX_BOTTOM, b_pos.x, b_pos.y);
```

Copyright 2025 YuviPep© All Rights Reserved

```
  if ((a_on && !b_on) || (!a_on && b_on)) {
    if (!flowStarted) {
      playFlowStartSound();
      flowStarted = true;
    }
    setLed(MATRIX_TOP, a_pos.x, a_pos.y, !a_on);
    setLed(MATRIX_BOTTOM, b_pos.x, b_pos.y, !b_on);
    // Softer drop sound
    tone(PIN_BUZZER, 600, 20);
    return true;
  } else {
    flowStarted = false; // Reset flow started flag when no particles move
  }
  return false;
}

int getGravity() {
  int tilt = digitalRead(PIN_TILT);
  return (tilt == LOW) ? 0 : 180;
}

int getTopMatrix() {
  return (getGravity() == 0) ? MATRIX_TOP : MATRIX_BOTTOM;
}

int getBottomMatrix() {
  return (getGravity() == 0) ? MATRIX_BOTTOM : MATRIX_TOP;
}

void resetTime() {
  for (byte i = 0; i < 2; i++) {
    lc.clearDisplay(i);
    for (byte x = 0; x < 8; x++) {
      for (byte y = 0; y < 8; y++) {
        ledState[i][x][y] = false;
        lc.setLed(i, y, x, false);
      }
    }
  }
  fill(getTopMatrix(), 60);
}
```

Copyright 2025 YuviPep© All Rights Reserved

```
void playStartupSound() {
  // Play a pleasant ascending melody
  int melody[] = {523, 659, 784}; // C5, E5, G5
  int durations[] = {100, 100, 150};
  for (int i = 0; i < 3; i++) {
    tone(PIN_BUZZER, melody[i], durations[i]);
    delay(durations[i] + 50);
    noTone(PIN_BUZZER);
  }
}

void playFlowStartSound() {
  // Sound for when particles start flowing after tilt
  int melody[] = {659, 784, 880}; // E5, G5, A5
  int durations[] = {80, 80, 120};
  for (int i = 0; i < 3; i++) {
    tone(PIN_BUZZER, melody[i], durations[i]);
    delay(durations[i] + 50);
    noTone(PIN_BUZZER);
  }
}

void playCompletionSound() {
  // Play a triumphant completion sequence
  int melody[] = {784, 659, 523, 659, 784}; // G5, E5, C5, E5, G5
  int durations[] = {100, 100, 100, 100, 200};
  for (int i = 0; i < 5; i++) {
    tone(PIN_BUZZER, melody[i], durations[i]);
    delay(durations[i] + 50);
    noTone(PIN_BUZZER);
  }
}

void setup() {
  pinMode(PIN_TILT, INPUT_PULLUP); // Enable internal pull-up on tilt sensor
  randomSeed(analogRead(A0));

  for (byte i = 0; i < 2; i++) {
    lc.shutdown(i, false);
    lc.setIntensity(i, 4);
    lc.clearDisplay(i);
  }
```

Copyright 2025 YuviPep© All Rights Reserved

```
  resetTime();
  playStartupSound(); // Play startup sound
}

void loop() {
  static unsigned long lastFrame = 0;
  static bool matrixTopFullSoundPlayed = false;
  static bool matrixBottomFullSoundPlayed = false;
  unsigned long now = millis();

  int old_rotation = current_rotation;
  int old_gravity = currentGravity;
  currentGravity = getGravity();
  current_rotation = (ROTATION_OFFSET + currentGravity) % 360;

  if (currentGravity != old_gravity) {
    remapState(old_rotation, current_rotation);
    matrixTopFullSoundPlayed = false; // Reset on tilt
    matrixBottomFullSoundPlayed = false; // Reset on tilt
  }

  // Fast, non-blocking frame update
  if (now - lastFrame >= DELAY_FRAME) {
    lastFrame = now;
    bool moved = updateMatrix();
    bool dropped = dropParticle();

    // Check if MATRIX_TOP is full
    if (countParticles(MATRIX_TOP) == 60 && !matrixTopFullSoundPlayed) {
      playCompletionSound();
      matrixTopFullSoundPlayed = true;
    }

    // Check if MATRIX_BOTTOM is full
    if (countParticles(MATRIX_BOTTOM) == 60 && !matrixBottomFullSoundPlayed) {
      playCompletionSound();
      matrixBottomFullSoundPlayed = true;
    }

    // Reset flags when particles move to allow sound on next full event
    if (dropped) {
      matrixTopFullSoundPlayed = false;
```

Copyright 2025 YuviPep© All Rights Reserved

```
      matrixBottomFullSoundPlayed = false;
    }
  }
}
```

Future Scope:

- Integrate a **real-time clock (RTC)** to make it a functional timer.
- Include **Bluetooth control** to adjust time duration or LED speed.
- Add **RGB matrices** for colorful sand flow effects.

Copyright 2025 YuviPep© All Rights Reserved